Falling enemies - classes and groups

Goal: By the end you can write a pygame.sprite.Sprite class, create many enemies from it and handle them together with a Group.
Before you start:

- Lesson 3 - the UFO that shoots.
- Knowing what a function is (def).

To copy the code: when you copy from a PDF, many viewers drop the spaces at the start of each line, and in Python those spaces matter. Every step is also a `.py` file in the same folder as this guide: open it and copy from there.

Step by step

Falling enemies: classes and groups

It is time for the enemies, and there will be many. To avoid repeating code, you will write a class: a mold for making enemies. Each enemy will have its own position and its own speed. And a pygame group will move and draw all of them together.

Step 2

First, next to the other images, load the enemy's, just like the UFO's, with convert_alpha to keep its transparency. You load it only once, up here, and all the enemies will share it. Now write a class: a mold that bundles an enemy's data and what it can do. It is called Enemy, capitalized, as is customary for classes. Inside the parentheses goes pygame.sprite.Sprite: your class inherits, meaning it gets for free what a sprite already knows. __init__ is a special function, which runs by itself every time you create an enemy. self is that specific enemy: whatever you store on self belongs to it alone. The line super().__init__() sets up the inherited part, and it always goes first. Then you give it its image and its Rect, in self.image and self.rect: those exact names are what the group looks for to draw it.

Your `game.py` now looks like this — highlighted lines are new or changed (file `step-1.py`):

```
import pygame

pygame.init()
screen = pygame.display.set_mode((800, 600))
pygame.display.set_caption("Most Wanted UFO Mini")
clock = pygame.time.Clock()

background = pygame.image.load("assets/background_4.png").convert()
player_image = pygame.image.load("assets/player_1.png").convert_alpha()
bullet_image = pygame.image.load("assets/Bullet.gif").convert_alpha()
enemy_image = pygame.image.load("assets/enemy_1.png").convert_alpha()
shoot_sound = pygame.mixer.Sound("assets/shoot.wav")
```

```
class Enemy(pygame.sprite.Sprite):
    def __init__(self):
        super().__init__()
        self.image = enemy_image
        self.rect = self.image.get_rect()


player = player_image.get_rect(midbottom=(400, 590))
speed = 6
bullets = []
cooldown = 0

running = True
while running:
    for event in pygame.event.get():
        if event.type == pygame.QUIT:
            running = False

    keys = pygame.key.get_pressed()
    if keys[pygame.K_LEFT]:
        player.x -= speed
    if keys[pygame.K_RIGHT]:
        player.x += speed
    player.clamp_ip(screen.get_rect())

    if cooldown > 0:
        cooldown -= 1
    if keys[pygame.K_SPACE] and cooldown == 0:
        bullets.append(bullet_image.get_rect(midbottom=player.midtop))
        shoot_sound.play()
        cooldown = 15

    for bullet in bullets:
        bullet.y -= 10
    bullets = [b for b in bullets if b.bottom > 0]

    screen.blit(background, (0, 0))
    for bullet in bullets:
        screen.blit(bullet_image, bullet)
    screen.blit(player_image, player)
    pygame.display.flip()
    clock.tick(60)

pygame.quit()
```

Step 3

At the very top, import random, Python's module for picking random numbers. By convention, imports go at the beginning of the file, before anything else. At the end of __init__, add self.reset(). A method is a function inside the class; you write reset right below, and you call it here so each new enemy takes its place as soon as it is created. reset receives self, like every method. It puts the enemy's bottom at a random x between twenty and seven hundred eighty. Its y is random too, between minus five hundred and minus forty: negative, so above the screen. That way they do not all arrive at once. It also gives it a speed of one or two. Since each one keeps its own speed on self, they will not all fall the same way. And update moves the enemy down as many pixels as its speed says. If

its top edge goes past six hundred, it has left through the bottom: it calls reset again and starts over at the top. Nobody calls update yet; the group will do that, every frame.

Your `game.py` now looks like this — highlighted lines are new or changed (file `step-2.py`):

```
import random

import pygame

pygame.init()
screen = pygame.display.set_mode((800, 600))
pygame.display.set_caption("Most Wanted UFO Mini")
clock = pygame.time.Clock()

background = pygame.image.load("assets/background_4.png").convert()
player_image = pygame.image.load("assets/player_1.png").convert_alpha()
bullet_image = pygame.image.load("assets/Bullet.gif").convert_alpha()
enemy_image = pygame.image.load("assets/enemy_1.png").convert_alpha()
shoot_sound = pygame.mixer.Sound("assets/shoot.wav")


class Enemy(pygame.sprite.Sprite):
    def __init__(self):
        super().__init__()
        self.image = enemy_image
        self.rect = self.image.get_rect()
        self.reset()

    def reset(self):
        self.rect.midbottom = (random.randint(20, 780), random.randint(-500, -40))
        self.speed = random.randint(1, 2)

    def update(self):
        self.rect.y += self.speed
        if self.rect.top > 600:
            self.reset()


player = player_image.get_rect(midbottom=(400, 590))
speed = 6
bullets = []
cooldown = 0

running = True
while running:
    for event in pygame.event.get():
        if event.type == pygame.QUIT:
            running = False

    keys = pygame.key.get_pressed()
    if keys[pygame.K_LEFT]:
        player.x -= speed
    if keys[pygame.K_RIGHT]:
        player.x += speed
    player.clamp_ip(screen.get_rect())

    if cooldown > 0:
        cooldown -= 1
```

```
    if keys[pygame.K_SPACE] and cooldown == 0:
        bullets.append(bullet_image.get_rect(midbottom=player.midtop))
        shoot_sound.play()
        cooldown = 15

    for bullet in bullets:
        bullet.y -= 10
    bullets = [b for b in bullets if b.bottom > 0]

    screen.blit(background, (0, 0))
    for bullet in bullets:
        screen.blit(bullet_image, bullet)
    screen.blit(player_image, player)
    pygame.display.flip()
    clock.tick(60)

pygame.quit()
```

Step 4

With the mold ready, make eight enemies, before the loop. This line works like the bullet list you filtered, but without brackets: it creates an Enemy eight times, and puts them all in a Group. The underscore is a variable you will not use: it just counts the repetitions. In the loop, after moving the bullets, enemies.update() calls the update of every enemy in the group. A single line moves all eight, each at its own speed. And enemies.draw(screen) draws them all, using each one's image and Rect. It goes right after the background, because if it came before, the background would cover them. If tomorrow you want twenty enemies, you just change the eight.

Your `game.py` now looks like this — highlighted lines are new or changed (file `step-3.py`):

```
import random

import pygame

pygame.init()
screen = pygame.display.set_mode((800, 600))
pygame.display.set_caption("Most Wanted UFO Mini")
clock = pygame.time.Clock()

background = pygame.image.load("assets/background_4.png").convert()
player_image = pygame.image.load("assets/player_1.png").convert_alpha()
bullet_image = pygame.image.load("assets/Bullet.gif").convert_alpha()
enemy_image = pygame.image.load("assets/enemy_1.png").convert_alpha()
shoot_sound = pygame.mixer.Sound("assets/shoot.wav")


class Enemy(pygame.sprite.Sprite):
    def __init__(self):
        super().__init__()
        self.image = enemy_image
        self.rect = self.image.get_rect()
        self.reset()

    def reset(self):
        self.rect.midbottom = (random.randint(20, 780), random.randint(-500, -40))
        self.speed = random.randint(1, 2)
```

```
    def update(self):
        self.rect.y += self.speed
        if self.rect.top > 600:
            self.reset()


player = player_image.get_rect(midbottom=(400, 590))
speed = 6
bullets = []
cooldown = 0
enemies = pygame.sprite.Group(Enemy() for _ in range(8))

running = True
while running:
    for event in pygame.event.get():
        if event.type == pygame.QUIT:
            running = False

    keys = pygame.key.get_pressed()
    if keys[pygame.K_LEFT]:
        player.x -= speed
    if keys[pygame.K_RIGHT]:
        player.x += speed
    player.clamp_ip(screen.get_rect())

    if cooldown > 0:
        cooldown -= 1
    if keys[pygame.K_SPACE] and cooldown == 0:
        bullets.append(bullet_image.get_rect(midbottom=player.midtop))
        shoot_sound.play()
        cooldown = 15

    for bullet in bullets:
        bullet.y -= 10
    bullets = [b for b in bullets if b.bottom > 0]
    enemies.update()

    screen.blit(background, (0, 0))
    enemies.draw(screen)
    for bullet in bullets:
        screen.blit(bullet_image, bullet)
    screen.blit(player_image, player)
    pygame.display.flip()
    clock.tick(60)

pygame.quit()
```

Step 5

Run the game. The enemies fall, each at its own speed, and when they leave through the bottom they reappear at the top. In this demo the UFO aims and shoots by itself, but look: the bullets go right through them. Nobody checks yet whether a bullet hits an enemy; that is the next lesson.

Try it:

```
python game.py
```

What you learned

Let's review. A class is a mold for creating objects. __init__ sets up each new enemy, and self is that enemy, with its own data. Sprite and Group already know how to draw and update many at once. And random makes every game different. Your exercise: give Enemy a sideways movement, so they fall in a zigzag. In the next lesson, the bullets finally hit.

- A class is a mold for creating objects
- self is each object with its data
- Sprite and Group already know how to draw and update
- random adds variety

Summary

- `class Enemy(pygame.sprite.Sprite):` defines a mold; `Enemy()` creates a new enemy and runs its `__init__`.
- `self` is the specific object: `self.rect` and `self.speed` belong to that enemy and no other.
- A `Sprite` needs `self.image` and `self.rect`; with those, `Group.draw` knows how to draw it.
- `pygame.sprite.Group(Enemy() for _ in range(8))` creates a group with 8 enemies.
- `enemies.update()` calls each enemy's `update` and `enemies.draw(screen)` draws them all.
- `random.randint(a, b)` gives a random integer between `a` and `b`, both included.

Exercise

Start from this lesson's `game.py`.

- Make each enemy, besides falling, move sideways at its own pace and bounce off the edges.

Hint: each enemy needs a new piece of data on `self`, chosen in `reset`.

- Bonus: change the 8 to 15 and see how the difficulty changes.

Solution

```
def reset(self):
    self.rect.midbottom = (random.randint(20, 780), random.randint(-500, -40))
    self.speed = random.randint(1, 2)
    self.drift = random.choice([-1, 1])

def update(self):
    self.rect.y += self.speed
    self.rect.x += self.drift
    if self.rect.left < 0 or self.rect.right > 800:
        self.drift = -self.drift
    if self.rect.top > 600:
        self.reset()
```

Art and sounds: Most Wanted UFO Classic, by Jose Daniel Soto, licensed CC BY 4.0.
A Magic Lab Studio course · magiclabstudio.com